Density Scene-Fitting Specification

Closed-loop adaptation of periodic HDR shells to named browser profiles

mdstats

2026-07-22

Contents

1 Purpose and status **1**

2 Ownership boundary **1**

3 Browser profiles **2**

4 Validated extraction prerequisite **2**

5 Backend-neutral shell geometry **2**

6 Scientific invariants **2**

7 Closed-loop algorithm **3**

8 Fit reports **3**

9 Failure policy **3**

10 Optional dependencies **3**

11 Focused validation **3**

12 External method **4**

1 Purpose and status

Module: `mdstats.plotting.density_scene_fit`
Status: normative and implemented through `mdstats 0.19.78a0`.

This module owns display-complexity adaptation after scientific scalar fields and initial meshes exist. It replaces estimate-and-abort behavior with an exact closed-loop controller.

2 Ownership boundary

The module owns:

- `BrowserMeshProfile`;
- backend-neutral `DensityShellGeometry`;
- shell and scene fit-attempt/result records;
- weighted target reallocation;
- bounded simplification retries;
- lower-resolution recontouring of the same scalar field;
- exact post-replication verification against `BrowserMeshBudget`.

It does not own density estimation, HDR threshold selection, QEM internals, raw mesh extraction, or Plotly serialization.

3 Browser profiles

Profile	Final density faces	Final density vertices	HTML bytes	Plotly traces
`compact`	300,000	225,000	40 MiB	72
`balanced`	600,000	450,000	72 MiB	96
`quality`	1,000,000	750,000	128 MiB	128
`custom`	caller supplied	caller supplied	caller supplied	caller supplied

`balanced` is the default. These are conservative package presets, not universal browser or GPU guarantees.

4 Validated extraction prerequisite

The scene fitter accepts only periodic meshes that already satisfy canonical seam pairing and edge-incidence requirements. Sparse extraction validates the globally welded tile surface before it creates `DensityShellGeometry`. Invalid tile-local presimplification is retried without local reduction; persistent extraction defects are repaired by bounded coarse recontouring or the declared non-mesh fallback.

The fitter therefore adapts display complexity, not malformed topology. Every QEM or recontour candidate produced inside this module is validated again before acceptance.

5 Backend-neutral shell geometry

```
DensityShellGeometry(
    shell_key: str,
    field: ScalarField3D,
    mass_fraction: float,
    contour_level: float,
    vertices_fractional: ndarray[n, 3],
    vertices_cartesian: ndarray[n, 3],
    faces: ndarray[m, 3],
    display_replication: int,
    visual_importance: float,
    minimum_faces: int,
    source_kind: str,
)
```

Dense and sparse contour paths must convert to this representation before fitting. Plotly objects must not enter the fitting layer.

6 Scientific invariants

Fitting may alter display geometry but must not alter:

- selected frames or weights;
- field values or normalization;
- Gaussian width or kernel cutoff;
- scientific logical grid;
- requested HDR mass fraction;
- scientific contour threshold;
- display cell or periodicity;
- topology-category identity.

Recontouring samples the already prepared scalar field at a coarser display grid and uses the same contour level. It is not a new density estimate.

7 Closed-loop algorithm

For shell s with canonical faces F_s and display multiplicity m_s, final compliance requires

$$\sum_s m_s F_s \le F_{\max}$$

with simultaneous vertex, trace, and HTML constraints.

The controller performs:

1. exact measurement of current post-replication usage;
2. initial target allocation from `density_scene_budget`;
3. periodic QEM simplification under strict fidelity limits;
4. browser-profile QEM retry with bounded relaxation;
5. compensated numerical targets when the simplifier overshoots;
6. lower-resolution recontouring when permitted;
7. periodic seam, triangle-incidence, finite-value, and degeneracy validation;
8. exact final budget evaluation before Plotly assembly.

Inner HDR shells receive greater visual importance than diffuse outer shells. The outer shell therefore absorbs more reduction when an additional scene-wide reduction is required.

8 Fit reports

```
DensityShellFitAttempt
DensityShellFitResult
DensitySceneFitReport
```

Every accepted or rejected attempt records method, requested target, resulting faces and vertices, validation status, and diagnostic metadata. An irreducible failure raises `BrowserMeshBudgetFailure` containing the scene report.

9 Failure policy

Failure is reserved for cases where no permitted periodic candidate can satisfy the selected profile. The controller must not fail merely because:

- the first simplification misses its target;
- one shell initially exceeds 250,000 faces in scene-controller mode;
- the initial scene is slightly above its profile budget.

The historical 301,838-, 314,640-, and 582,375-face cases must enter this controller.

10 Optional dependencies

QEM retries require `fast-simplification`. Recontouring requires scikit-image. Both are included by `mdstats[interactive]`. Missing optional dependencies must produce an explicit capability failure or a remaining permitted fallback; they must not silently skip final budget validation.

11 Focused validation

Tests must cover:

- profile coercion and custom budgets;

- dense and sparse shell normalization;
- target misses routed into fitting;
- exact post-replication face and vertex accounting;
- QEM retry and overshoot compensation;
- recontour fallback using an unchanged threshold;
- seam and incidence validation after every accepted candidate;
- deterministic repeated output;
- structured irreducible failure;
- the three historical count regressions.

12 External method

Mesh reduction uses quadric-error metrics following Garland and Heckbert, “Surface Simplification Using Quadric Error Metrics,” SIGGRAPH 1997, DOI: 10.1145/258734.258849. Contour extraction uses the Lewiner et al. topologically consistent marching-cubes method, *Journal of Graphics Tools* 8(2), 1–15 (2003), DOI: 10.1080/10867651.2003.10487582.

The weighted scene controller, retry ladder, periodic candidate validation, and profile policy are mdstats-specific integration designs.